

Programación declarativa

- Haskell •

Doble grado en Ingeniería Informática y Matemáticas

Universidad Complutense de Madrid

Óscar Martín — Curso 2026/27

14. Recursión y parámetros acumuladores

Factorial

```
factorial :: Int -> Int
factorial 0 = 1
factorial n = n * factorial (n-1)
```

- Evaluación paso a paso de `factorial 5`:

```
factorial 5
= 5 * factorial 4
= 5 * (4 * factorial 3)
= 5 * (4 * (3 * factorial 2))
= 5 * (4 * (3 * (2 * factorial 1)))
= 5 * (4 * (3 * (2 * (1 * factorial 0))))
= 5 * (4 * (3 * (2 * (1 * 1))))
= 5 * (4 * (3 * (2 * 1)))
= 5 * (4 * (3 * 2))
= 5 * (4 * 6)
= 5 * 24
= 120
```

- Quizá mejor:

```
factorial' n = product [1..n]
```

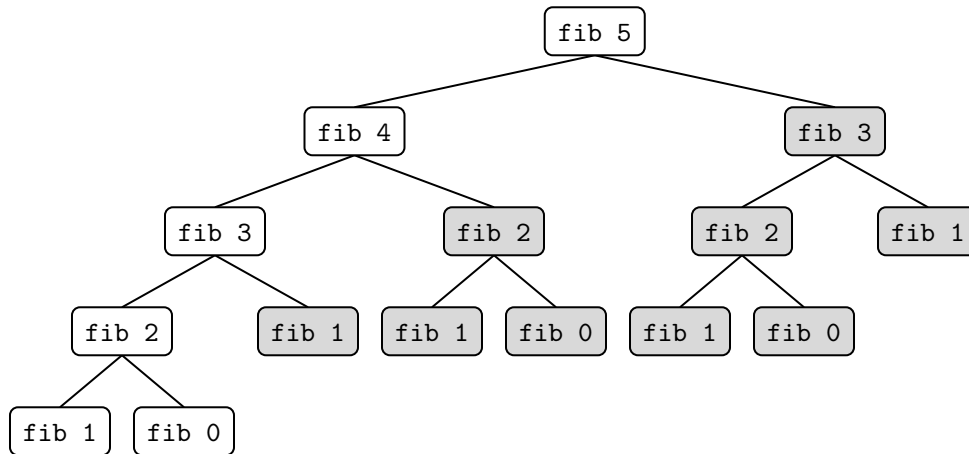
Fibonacci

```
fib :: Int -> Int
fib 0 = 1
fib 1 = 1
fib n = fib (n-1) + fib (n-2)
```

- Evaluación paso a paso de fib 5:

```
fib 5
= fib (5-1)          + fib (5-2)
= fib 4              + fib (5-2)
= (fib (4-1) + fib (4-2)) + fib (5-2)
= ...
= (3          + fib (4-2)) + fib (5-2)
= ...
= (3          + 2)        + fib (5-2)
= 5            + fib (5-2)
= 5            + fib 3
= ...
= 5            + 3
= 8
```

- La función fib tiene coste en tiempo exponencial en n porque repite cálculos.



```
fib' :: Int -> Int
fib' n = fibConMemoria n 1 1
  where fibConMemoria 0 x y = y
        fibConMemoria n x y = fibConMemoria (n-1) (x+y) x
```

- Dos acumuladores van llevando dos valores consecutivos de Fibonacci.
- Invariante: si la llamada inicial fue fib' n y ahora tenemos fibConMemoria m x y, se cumple que fibConMemoria m x y es el m-ésimo término de la sucesión de Fibonacci que empieza con x e y como términos iniciales.
- El coste es lineal en n, tanto en tiempo como en espacio.

Evaluación paso a paso de `fib' 5`:

```
fib' 5
= fibConMemoria 5      1      1
= fibConMemoria (5-1) (1+1)  1
= fibConMemoria 4      (1+1)  1
= fibConMemoria (4-1) ((1+1)+1) (1+1)
= fibConMemoria 3      <3>    <2>
= fibConMemoria (3-1) <5>      <3>
= ...
= fibConMemoria 1      <8>      <5>
= fibConMemoria (1-1) <13>     <8>
= fibConMemoria 0      <13>     <8>
= <8>
= 8
```

- Por simplificar, represento como `<5>` una expresión sin evaluar cuyo valor es `5`, como `((1+1)+1)+(1+1)`.
- Las expresiones intermedias crecen exponencialmente... pero Haskell tiene compartición de expresiones, como veremos más adelante.

Dar la vuelta a una lista

```
daLaVuelta :: [a] -> [a]
daLaVuelta [] = []
daLaVuelta (x:xs) = daLaVuelta xs ++ [x]
```

- Evaluación paso a paso de `daLaVuelta [1, 2, 3]`:

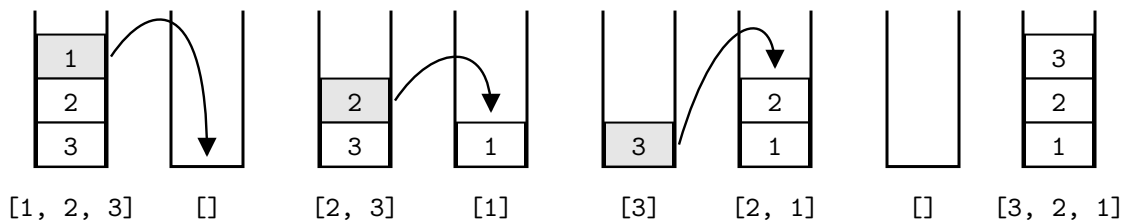
```
daLaVuelta [1, 2, 3]
= daLaVuelta [2, 3] ++ [1]
= (daLaVuelta [3] ++ [2]) ++ [1]
= ((daLaVuelta [] ++ [3]) ++ [2]) ++ [1]
= (([] ++ [3]) ++ [2]) ++ [1]
= ([3] ++ [2]) ++ [1]
= [3, 2] ++ [1]
= [3, 2, 1]
```

- La función `daLaVuelta` tiene coste cuadrático en la longitud de la lista, $O(n^2)$: hay n llamadas recursivas y en cada una la concatenación recorre $O(n)$ elementos.

```
daLaVuelta' :: [a] -> [a]
daLaVuelta' xs = daLaVueltaSobre xs []
  where daLaVueltaSobre [] ys = ys
        daLaVueltaSobre (x:xs) ys = daLaVueltaSobre xs (x:ys)
```

- Evaluación paso a paso de `daLaVuelta' [1, 2, 3]`:

```
daLaVuelta' [1, 2, 3]
= daLaVueltaSobre [1, 2, 3] []
= daLaVueltaSobre [2, 3] [1]
= daLaVueltaSobre [3] [2, 1]
= daLaVueltaSobre [] [3, 2, 1]
= [3, 2, 1]
```



- Invariante: si la llamada inicial fue `daLaVuelta' xs` y ahora tenemos `daLaVueltaSobre ys zs`, se cumple que `daLaVuelta' xs = daLaVuelta' ys ++ zs`.
- El coste es lineal en la longitud de la lista.

Recursión mutua

```
esPar, esImpar :: Int -> Bool
esPar 0 = True
esPar n = esImpar (n-1)
esImpar 0 = False
esImpar n = esPar (n-1)
```

Ejercicios

1. Define una función que inserte un elemento en una lista ordenada manteniendo el orden: `insertar 3 [1, 4, 5] ~> [1, 3, 4, 5]`. Úsala para definir una función para ordenar una lista con *ordenación por inserción*. Usa recursión explícita en las dos funciones.
2. Define una función para calcular el máximo común divisor de dos números enteros usando el algoritmo de Euclides. Define otra función para calcular el máximo común divisor de una lista de números enteros. Usa recursión explícita en las dos funciones.